

Death by a Thousand Queries.

Name _____

Date _____

Both of these shipped to production. **Every single query in them is fast** — indexed, sub-millisecond, nothing a query plan would complain about. Count the **queries**, not the rows; one round trip costs about **0.4 ms**. **Write your prediction down before you run anything.** Being wrong on paper is free.

1 The project side nav

PROJECTS		
Website Redesign	8	120
Mobile App	8	120
Q3 Marketing	8	120
... 37 more projects		

Workspace 1: **40 projects**, each with **8 collaborators** and **120 tasks**. Those two little numbers on every row are the whole problem.

```
projects = query("SELECT * FROM projects WHERE workspace_id=?")
for p in projects:
    collabs = query("SELECT * FROM collaborators WHERE project_id=?", p.id)
    for c in collabs:
        user = query("SELECT * FROM users WHERE id=?", c.user_id)
        tasks = query("SELECT * FROM tasks WHERE project_id=?", p.id)
        for t in tasks:
            who = query("SELECT * FROM users WHERE id=?", t.assigned_id)
```

queries = 1 + _____ + _____ + _____ + _____
total _____ at 0.4 ms each → _____ s

the one query that replaces all of it

It was tested with three projects and five tasks. It felt instant.

NOW RUN IT — PREDICT FIRST

```
curl -s localhost:5002/api/sidenav/slow | jq '{queries, ms}'
curl -s localhost:5002/api/sidenav/fast | jq '{queries, ms}'
```

you predicted _____ it actually did _____

2 The top three projects

MOST ACTIVE THIS WEEK	
1 API Migration	412
2 Website Redesign	388
3 Mobile App	355

Three rows on screen. The same **40 projects** behind them.

```
projects = query("SELECT id,name FROM projects WHERE workspace_id=?")
for p in projects:
    p.tasks = query("SELECT count(*) FROM tasks WHERE project_id=?", p.id)

for i in range(len(projects)):
    for j in range(len(projects)-1):
        a = query("SELECT count(*) FROM tasks WHERE project_id=?", projects[j].id)
        b = query("SELECT count(*) FROM tasks WHERE project_id=?", projects[j+1].id)
        if a < b: swap(projects, j, j+1)
show(projects[:3])
```

queries = 1 + _____ + _____
total _____ at 0.4 ms each → _____

the one query that replaces all of it

The task counts were **already fetched** in the first loop, then thrown away. **The database has a sort of its own, and an index. What would it have cost?**

NOW RUN IT — PREDICT FIRST

```
curl -s localhost:5002/api/top-projects/slow | jq '{queries, ms}'
curl -s localhost:5002/api/top-projects/fast | jq '{queries, ms}'
```

you predicted _____ it actually did _____

SCENARIO 2, AS THE WORKSPACE GROWS

PROJECTS	QUERIES	TIME
5		
10		
20		
40		

WHAT TO LOOK FOR IN YOUR OWN CODE

- a query inside a **for** loop
- a query inside a **sort comparator**
- fetching whole rows only to **count** them
- asking again for something you **already have**
- code that was only ever tested with **three** of something

Nothing here is a slow query. Every one of them is a fast query, asked far too many times. This is the bug a query plan **cannot show you** — and it is the one juniors ship most.

Setting it up

Needs **Docker Desktop** running. Nothing else to install — the database and the app both come up in containers.

1 Get it

```
git clone https://github.com/mchoi-altitude/\
training_slow-queries-lab
cd training_slow-queries-lab
```

2 Start it

```
docker compose up -d --build
```

First start seeds **200,000 users**, **40,000 projects** and **2,000,000 tasks**. About a minute. Later starts are instant.

3 Check it is alive

```
curl -s localhost:5002/api/stats
```

App on **:5002**, Postgres on **:5436** (lab/lab).
Workspace **1** is the one both endpoints use.

? If something is wrong

Port already in use — change it in docker-compose.yml.
Start over — docker compose down -v, then up again.
Done for the day — docker compose down.